

Department of Computer Science & Engineering

Software Engineering Laboratory

Course Code: 3CS511CC24

Practical – 9

Implementation of Formal Specification Using Z Notation

Project: Disaster Resource Coordination & Volunteer Allocation System

Subject: Software Engineering

Batch: C3

Roll No.: 23BCE183, 23BCE189, 23BCE178, 23BCE175

Contents

1	Aim	3
2	Theory	3
2.1	What is Formal Specification?	3
2.2	What is Z Notation?	3
2.3	Key Building Blocks of Z	3
2.3.1	Basic Types	3
2.3.2	Schema	3
2.3.3	Commonly Used Symbols	4
3	System Overview	4
4	Basic Type Declarations	5
5	State Schemas	5
5.1	User State	5
5.2	Disaster State	5
5.3	Resource State	6
5.4	Volunteer State	6
5.5	Allocation State	7
5.6	Task State	7
6	Operation Schemas	8
6.1	User Registration	8
6.2	User Login	8
6.3	Register Disaster Event	9
6.4	Update Disaster Status	9
6.5	Add Resource to Inventory	10
6.6	Register Volunteer	10
6.7	Verify Volunteer	11
6.8	Allocate Resource to Disaster	11
6.9	Deploy Volunteer to Disaster	12
6.10	Assign Task to Volunteer	13
6.11	Update Task Status	14
6.12	Generate Admin Report (Read-Only)	15
7	System Invariants	15
8	Summary of Operations	16
9	Conclusion	16

Aim

To write a formal specification for the Disaster Resource Coordination & Volunteer Allocation System using Z notation, covering all major modules of the system.

Theory

What is Formal Specification?

Formal specification means describing a system in a mathematically precise way. Instead of writing requirements in plain English, which can be ambiguous, we write them using mathematical notation. This removes guesswork and makes the expected behaviour of the system clear.

What is Z Notation?

Z (pronounced “Zed”) is a formal specification language based on set theory and predicate logic. It was developed at the University of Oxford. A Z specification describes *what* a system must do, not *how* it should do it.

Key Building Blocks of Z

Basic Types

These are the fundamental data types declared at the start.

$$[USER_ID, DISASTER_ID, RESOURCE_ID, VOLUNTEER_ID, TASK_ID]$$

This declares that the system works with these kinds of entities.

Schema

A schema is the main unit in Z. It has two parts separated by a horizontal line:

- **Declaration part** (above the line): lists the variables and their types.
- **Predicate part** (below the line): states the conditions that must always hold.

Commonly Used Symbols

Symbol	Name	Meaning
$\mathbb{P}$	Power set	Set of all subsets
$\rightarrow$	Total function	Every input maps to exactly one output
$\mapsto$	Injection	Different inputs give different outputs
$\rightharpoonup$	Partial function	Input may or may not have a mapped output
$\in$	Element of	Belongs to a set
$\notin$	Not element of	Does not belong to a set
$\subseteq$	Subset	All elements of one set are in another
$\cap$	Intersection	Common elements
$\cup$	Union	Combined elements
Δ	Delta	Schema allows state change
Ξ	Xi	Schema does not change the state
$\exists$	There exists	At least one value satisfies this
$\forall$	For all	Every value satisfies this
$\wedge$	And	Both conditions must hold
$\vee$	Or	At least one condition must hold
$\neg$	Not	Condition must not hold
' (prime)	Post-state	Value of variable after an operation
$\oplus$	Override	Updates one mapping, leaves rest unchanged
$?$	Input	Denotes an input to an operation
$!$	Output	Denotes an output from an operation
$\#$	Cardinality	Number of elements in a set

System Overview

The Disaster Resource Coordination & Volunteer Allocation System is a centralised web-based platform that integrates multiple disaster response activities into a single system. It replaces traditional manual coordination methods with a real-time digital solution that improves visibility and control.

The key modules specified in this practical are:

1. User Registration and Authentication
2. Disaster Event Registration and Management
3. Resource Inventory Management
4. Volunteer Registration and Verification
5. Resource and Volunteer Allocation
6. Task Assignment and Tracking
7. Administrative Reporting

Basic Type Declarations

All entities in the system are declared as basic types first.

$[USER_ID, DISASTER_ID, RESOURCE_ID, VOLUNTEER_ID, TASK_ID, ALLOCATION_ID]$

$ROLE ::= admin \mid authority \mid coordinator \mid volunteer \mid resource_provider \mid citizen$

$USER_STATUS ::= active \mid inactive \mid suspended$

$DISASTER_TYPE ::= flood \mid earthquake \mid cyclone \mid fire \mid landslide \mid other$

$SEVERITY ::= low \mid medium \mid high \mid critical$

$DISASTER_STATUS ::= ongoing \mid resolved \mid monitoring$

$RESOURCE_TYPE ::= food \mid medical \mid shelter \mid vehicle \mid equipment \mid other$

$VOLUNTEER_STATUS ::= registered \mid verified \mid deployed \mid available \mid inactive$

$TASK_STATUS ::= assigned \mid in_progress \mid completed \mid cancelled$

$ALLOC_STATUS ::= active \mid completed \mid cancelled$

State Schemas

User State

This schema describes all data the system stores for each registered user.

$UserState$ $registeredUsers : \mathbb{P} \ USER_ID$ $userRole : USER_ID \rightarrow ROLE$ $userStatus : USER_ID \rightarrow USER_STATUS$ $userEmail : USER_ID \hookrightarrow \mathbb{P} \ \mathbb{N}$
$dom(userRole) = registeredUsers$ $dom(userStatus) = registeredUsers$ $dom(userEmail) = registeredUsers$

The predicate section ensures every registered user has a role, status, and unique email. The injection on $userEmail$ ensures no two users share the same email address.

Disaster State

This schema describes all data the system maintains for each registered disaster event.

DisasterState

$$\begin{aligned} \text{registeredDisasters} &: \mathbb{P} \text{ DISASTER_ID} \\ \text{disasterType} &: \text{DISASTER_ID} \rightarrow \text{DISASTER_TYPE} \\ \text{disasterLocation} &: \text{DISASTER_ID} \rightarrow \mathbb{N} \\ \text{disasterSeverity} &: \text{DISASTER_ID} \rightarrow \text{SEVERITY} \\ \text{disasterStatus} &: \text{DISASTER_ID} \rightarrow \text{DISASTER_STATUS} \\ \text{reportedBy} &: \text{DISASTER_ID} \rightarrow \text{USER_ID} \end{aligned}$$

$$\begin{aligned} \text{dom}(\text{disasterType}) &= \text{registeredDisasters} \\ \text{dom}(\text{disasterLocation}) &= \text{registeredDisasters} \\ \text{dom}(\text{disasterSeverity}) &= \text{registeredDisasters} \\ \text{dom}(\text{disasterStatus}) &= \text{registeredDisasters} \\ \text{dom}(\text{reportedBy}) &= \text{registeredDisasters} \end{aligned}$$

Every registered disaster must have a type, location, severity, current status, and the user who reported it. No partial disaster records are permitted.

Resource State

This schema describes all data for resources available in the system inventory, such as food, medical kits, shelter materials, and vehicles.

ResourceState

$$\begin{aligned} \text{availableResources} &: \mathbb{P} \text{ RESOURCE_ID} \\ \text{resourceType} &: \text{RESOURCE_ID} \rightarrow \text{RESOURCE_TYPE} \\ \text{resourceQuantity} &: \text{RESOURCE_ID} \rightarrow \mathbb{N} \\ \text{resourceLocation} &: \text{RESOURCE_ID} \rightarrow \mathbb{N} \\ \text{resourceProvider} &: \text{RESOURCE_ID} \rightarrow \text{USER_ID} \end{aligned}$$

$$\begin{aligned} \text{dom}(\text{resourceType}) &= \text{availableResources} \\ \text{dom}(\text{resourceQuantity}) &= \text{availableResources} \\ \text{dom}(\text{resourceLocation}) &= \text{availableResources} \\ \text{dom}(\text{resourceProvider}) &= \text{availableResources} \\ \forall r &: \text{availableResources} \bullet \text{resourceQuantity}(r) \geq 0 \end{aligned}$$

A resource may have quantity zero (exhausted) but still remain in the registry. The provider must be a registered user with the *resource_provider* or *admin* role.

Volunteer State

This schema describes all data for registered volunteers, including their skills, availability, and verification status.

VolunteerState

$$\begin{aligned}
& \text{registeredVolunteers} : \mathbb{P} \text{ VOLUNTEER_ID} \\
& \text{volunteerUser} : \text{VOLUNTEER_ID} \rightarrow \text{USER_ID} \\
& \text{volunteerSkills} : \text{VOLUNTEER_ID} \rightarrow \mathbb{P} \mathbb{N} \\
& \text{volunteerStatus} : \text{VOLUNTEER_ID} \rightarrow \text{VOLUNTEER_STATUS} \\
& \text{volunteerLocation} : \text{VOLUNTEER_ID} \rightarrow \mathbb{N}
\end{aligned}$$

$$\begin{aligned}
& \text{dom}(\text{volunteerUser}) = \text{registeredVolunteers} \\
& \text{dom}(\text{volunteerSkills}) = \text{registeredVolunteers} \\
& \text{dom}(\text{volunteerStatus}) = \text{registeredVolunteers} \\
& \text{dom}(\text{volunteerLocation}) = \text{registeredVolunteers} \\
& \text{ran}(\text{volunteerUser}) \subseteq \text{registeredUsers}
\end{aligned}$$

The injection on *volunteerUser* ensures each volunteer profile links to exactly one user account and no user can have two volunteer profiles. All volunteer user IDs must correspond to registered users.

Allocation State

This schema describes allocations of resources or volunteers to specific disaster zones.

AllocationState

$$\begin{aligned}
& \text{allAllocations} : \mathbb{P} \text{ ALLOCATION_ID} \\
& \text{allocDisaster} : \text{ALLOCATION_ID} \rightarrow \text{DISASTER_ID} \\
& \text{allocResource} : \text{ALLOCATION_ID} \rightarrow \text{RESOURCE_ID} \\
& \text{allocVolunteer} : \text{ALLOCATION_ID} \rightarrow \text{VOLUNTEER_ID} \\
& \text{allocStatus} : \text{ALLOCATION_ID} \rightarrow \text{ALLOC_STATUS} \\
& \text{allocatedBy} : \text{ALLOCATION_ID} \rightarrow \text{USER_ID}
\end{aligned}$$

$$\begin{aligned}
& \text{dom}(\text{allocDisaster}) = \text{allAllocations} \\
& \text{dom}(\text{allocStatus}) = \text{allAllocations} \\
& \text{dom}(\text{allocatedBy}) = \text{allAllocations} \\
& \forall a : \text{allAllocations} \bullet \\
& \quad \mathbb{P} a \in \text{dom}(\text{allocResource}) \vee a \in \text{dom}(\text{allocVolunteer})
\end{aligned}$$

An allocation must link to a specific disaster and must contain at least one resource or one volunteer. Partial functions allow either field to be absent, but not both simultaneously.

Task State

This schema describes tasks assigned to volunteers or field coordinators during a disaster response.

TaskState

$$\begin{aligned}
&allTasks : \mathbb{P} \text{ TASK_ID} \\
&taskDisaster : TASK_ID \rightarrow DISASTER_ID \\
&taskVolunteer : TASK_ID \rightarrow VOLUNTEER_ID \\
&taskDescription : TASK_ID \rightarrow \mathbb{N} \\
&taskStatus : TASK_ID \rightarrow TASK_STATUS \\
&taskAssignedBy : TASK_ID \rightarrow USER_ID
\end{aligned}$$

$$\begin{aligned}
&\text{dom}(taskDisaster) = allTasks \\
&\text{dom}(taskDescription) = allTasks \\
&\text{dom}(taskStatus) = allTasks \\
&\text{dom}(taskAssignedBy) = allTasks
\end{aligned}$$

A task may be unassigned to any specific volunteer (partial function) but must always be associated with a disaster event and have a description, status, and the coordinator who created it.

Operation Schemas

User Registration

A new user registers with a unique ID. The state changes to add this user.

RegisterUser

$$\begin{aligned}
&\Delta UserState \\
&newUser? : USER_ID \\
&role? : ROLE
\end{aligned}$$

$$\begin{aligned}
&newUser? \notin registeredUsers \\
®isteredUsers' = registeredUsers \cup \{newUser?\} \\
&userRole' = userRole \cup \{newUser? \mapsto role?\} \\
&userStatus' = userStatus \cup \{newUser? \mapsto active\}
\end{aligned}$$

The Δ symbol says the state will change. The condition $newUser? \notin registeredUsers$ prevents duplicate registration. Primed variables represent the state after the operation. All new users are set to *active* status by default.

User Login

This operation only checks the state; it does not change it.

LoginUser

$$\begin{aligned}
&\Xi UserState \\
&uid? : USER_ID \\
&result! : USER_STATUS
\end{aligned}$$

$$\begin{aligned}
&uid? \in registeredUsers \\
&result! = userStatus(uid?)
\end{aligned}$$

Ξ means the state stays the same. The user must already be registered, and the system simply returns their status.

Register Disaster Event

An authorised user (admin or authority) registers a new disaster event with all required details.

RegisterDisaster $\Delta\text{DisasterState}$ $\Xi\text{UserState}$ $did? : \text{DISASTER_ID}$ $dtype? : \text{DISASTER_TYPE}$ $loc? : \mathbb{N}$ $severity? : \text{SEVERITY}$ $reporter? : \text{USER_ID}$
$reporter? \in \text{registeredUsers}$ $\text{userRole}(reporter?) \in \{\text{admin}, \text{authority}\}$ $did? \notin \text{registeredDisasters}$ $\text{registeredDisasters}' = \text{registeredDisasters} \cup \{did?\}$ $\text{disasterType}' = \text{disasterType} \cup \{did? \mapsto dtype?\}$ $\text{disasterLocation}' = \text{disasterLocation} \cup \{did? \mapsto loc?\}$ $\text{disasterSeverity}' = \text{disasterSeverity} \cup \{did? \mapsto severity?\}$ $\text{disasterStatus}' = \text{disasterStatus} \cup \{did? \mapsto \text{ongoing}\}$ $\text{reportedBy}' = \text{reportedBy} \cup \{did? \mapsto reporter?\}$

Only users with *admin* or *authority* roles can register a disaster. The initial status of every new disaster is automatically set to *ongoing*.

Update Disaster Status

An authorised user updates the status of an existing disaster event (e.g., from *ongoing* to *resolved*).

$\text{UpdateDisasterStatus}$ $\Delta\text{DisasterState}$ $\Xi\text{UserState}$ $did? : \text{DISASTER_ID}$ $\text{newStatus?} : \text{DISASTER_STATUS}$ $\text{updater?} : \text{USER_ID}$
$did? \in \text{registeredDisasters}$ $\text{updater?} \in \text{registeredUsers}$ $\text{userRole}(\text{updater?}) \in \{\text{admin}, \text{authority}, \text{coordinator}\}$ $\text{disasterStatus}(did?) \neq \text{resolved}$ $\text{disasterStatus}' = \text{disasterStatus} \oplus \{did? \mapsto \text{newStatus?}\}$

$\oplus$ is the overriding operator – it updates only the status of the specified disaster, leaving all other disaster records unchanged. A resolved disaster cannot be modified again.

Add Resource to Inventory

A resource provider or administrator adds a new resource to the system inventory.

$AddResource$	
$\Delta ResourceState$ $\Xi UserState$ $rid? : RESOURCE_ID$ $rtype? : RESOURCE_TYPE$ $qty? : \mathbb{N}$ $loc? : \mathbb{N}$ $provider? : USER_ID$	
$provider? \in registeredUsers$ $userRole(provider?) \in \{resource_provider, admin\}$ $rid? \notin availableResources$ $qty? \geq 0$ $availableResources' = availableResources \cup \{rid?\}$ $resourceType' = resourceType \cup \{rid? \mapsto rtype?\}$ $resourceQuantity' = resourceQuantity \cup \{rid? \mapsto qty?\}$ $resourceLocation' = resourceLocation \cup \{rid? \mapsto loc?\}$ $resourceProvider' = resourceProvider \cup \{rid? \mapsto provider?\}$	

Only *resource_provider* or *admin* roles may add resources. The resource ID must be new to the system.

Register Volunteer

A user with the volunteer role registers as a volunteer by providing their skills and location.

$RegisterVolunteer$	
$\Delta VolunteerState$ $\Xi UserState$ $vid? : VOLUNTEER_ID$ $uid? : USER_ID$ $skills? : \mathbb{P} \mathbb{N}$ $loc? : \mathbb{N}$	
$uid? \in registeredUsers$ $userRole(uid?) = volunteer$ $vid? \notin registeredVolunteers$ $uid? \notin \text{ran}(volunteerUser)$ $registeredVolunteers' = registeredVolunteers \cup \{vid?\}$ $volunteerUser' = volunteerUser \cup \{vid? \mapsto uid?\}$ $volunteerSkills' = volunteerSkills \cup \{vid? \mapsto skills?\}$ $volunteerStatus' = volunteerStatus \cup \{vid? \mapsto registered\}$ $volunteerLocation' = volunteerLocation \cup \{vid? \mapsto loc?\}$	

$uid? \notin \text{ran}(\text{volunteerUser})$ ensures one user cannot create two volunteer profiles. New volunteers start with *registered* status and must be verified before deployment.

Verify Volunteer

An administrator or coordinator verifies a registered volunteer, marking them as ready for deployment.

VerifyVolunteer $\Delta \text{VolunteerState}$ $\Xi \text{UserState}$ $vid? : \text{VOLUNTEER_ID}$ $verifier? : \text{USER_ID}$
$vid? \in \text{registeredVolunteers}$ $\text{volunteerStatus}(vid?) = \text{registered}$ $verifier? \in \text{registeredUsers}$ $\text{userRole}(verifier?) \in \{\text{admin}, \text{authority}, \text{coordinator}\}$ $\text{volunteerStatus}' = \text{volunteerStatus} \oplus \{vid? \mapsto \text{verified}\}$

Only unverified (*registered*) volunteers can be verified. This gate prevents re-verification of already deployed or active volunteers.

Allocate Resource to Disaster

A coordinator or administrator allocates a quantity of an available resource to an ongoing disaster zone.

$\Delta AllocationState$ $\Delta ResourceState$ $\Xi DisasterState$ $\Xi UserState$ $aid? : ALLOCATION_ID$ $did? : DISASTER_ID$ $rid? : RESOURCE_ID$ $qty? : \mathbb{N}$ $allocator? : USER_ID$	$allocator? \in registeredUsers$ $userRole(allocator?) \in \{admin, authority, coordinator\}$ $did? \in registeredDisasters$ $disasterStatus(did?) = ongoing$ $rid? \in availableResources$ $resourceQuantity(rid?) \geq qty?$ $qty? > 0$ $aid? \notin allAllocations$ $allAllocations' = allAllocations \cup \{aid?\}$ $allocDisaster' = allocDisaster \cup \{aid? \mapsto did?\}$ $allocResource' = allocResource \cup \{aid? \mapsto rid?\}$ $allocStatus' = allocStatus \cup \{aid? \mapsto active\}$ $allocatedBy' = allocatedBy \cup \{aid? \mapsto allocator?\}$ $resourceQuantity' = resourceQuantity \oplus \{rid? \mapsto resourceQuantity(rid?) - qty?\}$
---	---

Resources can only be allocated to *ongoing* disasters. The available quantity must be sufficient. Allocation atomically decrements the inventory quantity using $\oplus$.

Deploy Volunteer to Disaster

A coordinator deploys a verified, available volunteer to an ongoing disaster zone.

Deploy Volunteer $\Delta AllocationState$ $\Delta VolunteerState$ $\Xi DisasterState$ $\Xi UserState$ $aid? : ALLOCATION_ID$ $did? : DISASTER_ID$ $vid? : VOLUNTEER_ID$ $allocator? : USER_ID$ $allocator? \in registeredUsers$ $userRole(allocator?) \in \{admin, authority, coordinator\}$ $did? \in registeredDisasters$ $disasterStatus(did?) = ongoing$ $vid? \in registeredVolunteers$ $volunteerStatus(vid?) = verified$ $aid? \notin allAllocations$ $allAllocations' = allAllocations \cup \{aid?\}$ $allocDisaster' = allocDisaster \cup \{aid? \mapsto did?\}$ $allocVolunteer' = allocVolunteer \cup \{aid? \mapsto vid?\}$ $allocStatus' = allocStatus \cup \{aid? \mapsto active\}$ $allocatedBy' = allocatedBy \cup \{aid? \mapsto allocator?\}$ $volunteerStatus' = volunteerStatus \oplus \{vid? \mapsto deployed\}$

Only *verified* volunteers can be deployed. Deployment atomically updates the volunteer's status to *deployed* using $\oplus$, preventing double-deployment.

Assign Task to Volunteer

A coordinator assigns a specific task to a deployed volunteer in the context of a disaster.

AssignTask $\Delta TaskState$ $\Xi UserState$ $\Xi VolunteerState$ $\Xi DisasterState$ $tid? : TASK_ID$ $did? : DISASTER_ID$ $vid? : VOLUNTEER_ID$ $desc? : \mathbb{N}$ $assignedBy? : USER_ID$ $assignedBy? \in registeredUsers$ $userRole(assignedBy?) \in \{admin, authority, coordinator\}$ $did? \in registeredDisasters$ $disasterStatus(did?) = ongoing$ $vid? \in registeredVolunteers$ $volunteerStatus(vid?) = deployed$ $tid? \notin allTasks$ $allTasks' = allTasks \cup \{tid?\}$ $taskDisaster' = taskDisaster \cup \{tid? \mapsto did?\}$ $taskVolunteer' = taskVolunteer \cup \{tid? \mapsto vid?\}$ $taskDescription' = taskDescription \cup \{tid? \mapsto desc?\}$ $taskStatus' = taskStatus \cup \{tid? \mapsto assigned\}$ $taskAssignedBy' = taskAssignedBy \cup \{tid? \mapsto assignedBy?\}$

Tasks can only be assigned to volunteers already *deployed* in the field. The initial status of every new task is *assigned*.

Update Task Status

A coordinator or volunteer updates the progress status of an existing task.

UpdateTaskStatus $\Delta TaskState$ $\Xi UserState$ $tid? : TASK_ID$ $newStatus? : TASK_STATUS$ $updater? : USER_ID$ $tid? \in allTasks$ $updater? \in registeredUsers$ $userRole(updater?) \in \{admin, authority, coordinator, volunteer\}$ $taskStatus(tid?) \neq completed$ $taskStatus(tid?) \neq cancelled$ $taskStatus' = taskStatus \oplus \{tid? \mapsto newStatus?\}$

completed or *cancelled* tasks are terminal – their status cannot be changed further. $\oplus$ updates only the specified task's status, leaving all others unchanged.

Generate Admin Report (Read-Only)

An administrator queries the system to generate a summary of active disasters, resource usage, and volunteer deployment. No state changes occur.

<i>AdminReport</i>
Ξ <i>DisasterState</i> Ξ <i>ResourceState</i> Ξ <i>VolunteerState</i> Ξ <i>TaskState</i> Ξ <i>UserState</i> <i>totalDisasters!</i> : $\mathbb{N}$ <i>activeDisasters!</i> : $\mathbb{N}$ <i>totalResources!</i> : $\mathbb{N}$ <i>deployedVolunteers!</i> : $\mathbb{N}$ <i>completedTasks!</i> : $\mathbb{N}$ <i>activeUsers!</i> : $\mathbb{N}$
<i>totalDisasters!</i> = # <i>registeredDisasters</i> <i>activeDisasters!</i> = # $\{d : \text{registeredDisasters} \mid \text{disasterStatus}(d) = \text{ongoing}\}$ <i>totalResources!</i> = # <i>availableResources</i> <i>deployedVolunteers!</i> = # $\{v : \text{registeredVolunteers} \mid \text{volunteerStatus}(v) = \text{deployed}\}$ <i>completedTasks!</i> = # $\{t : \text{allTasks} \mid \text{taskStatus}(t) = \text{completed}\}$ <i>activeUsers!</i> = # $\{u : \text{registeredUsers} \mid \text{userStatus}(u) = \text{active}\}$

All five state schemas are marked Ξ – this operation is entirely read-only. The # operator counts the number of elements in each filtered set. No state changes here.

System Invariants

Invariants are conditions that must hold at all times, regardless of which operation runs.

- Every disaster must have been registered by a user with *admin* or *authority* role.
- Resources can only be allocated to *ongoing* disasters – not *resolved* or *monitoring* ones.
- A volunteer must be *verified* before they can be deployed or assigned tasks.
- Each volunteer profile is linked to exactly one user account (enforced by injection).
- Resource quantity allocated cannot exceed the available quantity in the inventory.
- Tasks in *completed* or *cancelled* state are terminal and cannot be further modified.
- Every allocation must be associated with at least one resource or one volunteer.
- Only users with appropriate roles (admin, authority, coordinator) can perform allocation and registration operations.

Formally, the core allocation invariant is:

$$\begin{aligned} & \forall a : allAllocations \bullet allocDisaster(a) \in registeredDisasters \\ & \mathcal{P} \wedge disasterStatus(allocDisaster(a)) = ongoing \\ & [6pt] \forall v : registeredVolunteers \bullet volunteerUser(v) \in registeredUsers \end{aligned}$$

Summary of Operations

Operation	Type	State Changed	Key Condition
RegisterUser	Δ UserState	UserState	User must not already exist
LoginUser	Ξ None	No change	User must be registered
RegisterDisaster	Δ DisasterState	DisasterState	Caller must be admin/authority
UpdateDisasterStatus	Δ DisasterState	DisasterState	Disaster must not be resolved
AddResource	Δ ResourceState	ResourceState	Caller must be provider/admin
RegisterVolunteer	Δ VolunteerState	VolunteerState	User must have volunteer role
VerifyVolunteer	Δ VolunteerState	VolunteerState	Volunteer must be in registered status
AllocateResource	Δ Alloc + Resource	Alloc, Resource	Disaster ongoing; qty available
DeployVolunteer	Δ Alloc + Volunteer	Alloc, Volunteer	Volunteer must be verified
AssignTask	Δ TaskState	TaskState	Volunteer must be deployed
UpdateTaskStatus	Δ TaskState	TaskState	Task must not be terminal
AdminReport	Ξ None	No change	Read-only, all Ξ schemas

Conclusion

In this practical, formal specifications were written for the Disaster Resource Coordination & Volunteer Allocation System using Z notation. Each module – user management, disaster event registration, resource inventory, volunteer management, allocation, task tracking, and administration – was described using state schemas and operation schemas.

The Z specifications make the expected system behaviour precise and unambiguous. They clearly state what conditions must hold before and after each operation, and they define what data the system tracks at any point. This level of clarity helps avoid misunderstandings during design and development.

The allocation engine – where coordinators can match available resources and verified volunteers to ongoing disaster zones – was formally captured through the *AllocateResource* and *DeployVolunteer* operations, with appropriate role, status, and quantity checks. The task tracking module was captured through *AssignTask* and *UpdateTaskStatus*, ensuring tasks are only given to deployed volunteers and cannot regress from terminal states.